

TemaFinale26 – Sprint 1 (v1)

Introduction

Questo documento (template del corso) e' focalizzato sull'**analisi del problema** del **cargoservice**. Costruisce un **modello logico** della soluzione — *di che tipo* sono i componenti, *come* interagiscono e *quale* comportamento ci si attende — **senza** fissare i dettagli di realizzazione (codice qak concreto, classi, trasporto), che sono Progetto/Development. Requisiti e analisi dei requisiti sono in [Sprint 0 \(v1\)](#).

Goal dello Sprint 1 (sottoinsieme del testo del committente): analizzare e modellare il **core business** — *"the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot"*, con ritorno a HOME e sistema pronto per una nuova richiesta.

Requirements

Il testo esatto del committente e' riportato in [Sprint 0 \(v1\) – Requirements](#). In sintesi, per il core: ricevuta una richiesta valida (pushbutton), il cargoservice riserva uno slot e usa il cargorobot per portare il container IOPort→slot5(marcatura)→slot riservato, poi torna a HOME.

Requirement analysis

Le entita' del dominio (Container, Slot, Hold, IOPort, sensor, marker, LED, cargorobot), la formalizzazione dell'area e le user stories sono in [Sprint 0 \(v1\) – Requirement analysis](#). Qui se ne analizza la **natura come componenti software** e le **interazioni**.

Problem analysis

Perche' qak (QActor): motivazione

La scelta del modello a **attori** non e' arbitraria: il cargoservice e' insieme **proattivo** (guida una sessione di carico in piu' passi mantenendo uno stato) e **reattivo** (pushbutton, sensor, marker, timeout). Questa combinazione «macchina a stati + reazione a messaggi/eventi + temporizzazione» e' cio' che QActor rende di prima classe:

Esigenza del problema	Costrutto QActor
comportamento a fasi con memoria	State/Transition + variabili
richiesta con risposta (pushbutton, robot)	whenRequest/replyTo ; request/whenReply
notifiche asincrone (sensor, marker)	whenEvent
guasto che interrompe da qualunque stato	whenInterruptEvent

timeout dei 30 s	whenTime
------------------	----------

Di che tipo sono i componenti? (pojo / service / attore)

- **POJO**: oggetto passivo, invocato in modo sincrono; incapsula dati/stato.
- **Service**: espone operazioni e *risponde* a richieste; reattivo, non ha piano proprio.
- **Attore (QActor)**: autonomo, con stato, proattivo + reattivo, comunica solo per messaggi.

Entita'	Discussione	Natura
cargoservice	guida una sessione in piu' passi e reagisce a eventi: un pojo/service non basta	attore
cargorobot	fornito; raggiunge una posizione su comando (vedi sotto)	service esterno
sensor dell'IOPort	fornisce la distanza D; il sonar e' una sua parte ; natura da chiarire	sorgente di notifiche — <i>aperto</i>
IOPort	pushbutton (sorgente di richiesta = requisito) + display (output; forse web-gui)	pushbutton + display
marker	segnala il completamento	service/attore (evento)
LED	riflette lo stato engaged	indicatore di stato
Hold	stato del dominio, invocato in modo sincrono	POJO

Quale software per il cargorobot, e perche'

Il committente fornisce **piu' componenti** per il DDR: RobotObj26 (POJO, mosse elementari: troppo basso livello), RobotService26 (service, ancora a livello di mosse), robotsmart (service con moverobot(X,Y) orientato all'obiettivo + A*). Il problema chiede di **raggiungere una posizione** (IOPort, slot5, slot, HOME): e' l'astrazione di moverobot(X,Y). Si sceglie quindi, **motivandolo**, robotsmart come service esterno di cui il cargoservice e' client (come boundaryworker usa robotservice). La versione esatta e' rifinitura di Progetto.

Modello delle interazioni

Ruoli, flussi e *natura* di ciascuno (non una semplice lista di messaggi):

```

    richiesta di carico / risposta
customer <-----> CARGOSERVICE <----> cargorobot (raggiungi posizione / esito)
    (pushbutton + display)      | ^
    presenza container / guasto sensor --+ +-- marker (marcatura completata)
                                |
                                stato hold (display/web-gui) ; engaged (LED)

```

Flusso	Natura	Altitudine
pushbutton → cargoservice	request/reply	requisito
sensor → cargoservice	notifica → evento (alt.: dispatch)	analisi (motivata)
marker → cargoservice	notifica "fatto" → evento	analisi
cargoservice → cargorobot	request/reply (moverobot)	contratto robotsmart
cargoservice → display/LED	output di stato	progetto / aperto

Il **trasporto** (TCP/CoAP/MQTT) non e' deciso qui: e' Progetto.

Formalizzazione (qak) delle interazioni di base

L'interazione request/reply del pushbutton (un **requisito**) e il contratto del cargorobot si formalizzano come **interfacce di messaggi** (non realizzazione):

```
Request loadrequest : loadrequest()
Reply answer       : answer(ESP) for loadrequest   ESP = reserved(SLOT)|retrylater|reject

Request moverobot   : moverobot(TARGETX, TARGETY, STEPTIME) // contratto di robotsmart
Reply moverobotdone : moverobotok(ARG)                  for moverobot
Reply moverobotfailed: moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

La FSM completa (stati, withobj, codice) e' **realizzazione**: vedi la sezione *Project* e il modello [cargoservice26.qak](#).

Comportamento atteso (livello di problema)

disponibile -> (richiesta valida) engaged: riserva slot, LED on
-> container all'IOPort -> cargorobot: IOPort -> slot5 (marcatura) -> slot riservato
-> ritorno a HOME -> disponibile
Vincoli: una sola richiesta per volta; stiva piena -> reject; IOPort occupato/Out of service -> retrylater.

Core business e strategia incrementale

Iterazione	Cosa realizza
Sprint 0 (core)	flusso nominale (container assunto presente); display/LED come log
Sprint 1	sensor dell'IOPort + timeout 30s
Sprint 2	Out of service + display web-gui

Decisioni e domande aperte

Tema	Altitudine	Stato
deposito = il robot raggiunge la posizione (no attuatore di presa)	analisi	confermato
cargorobot = robotsmart	analisi (motivata)	versione → Progetto
natura del sensor / del display (web-gui?)	analisi	aperta
trasporto (MQTT/TCP/CoAP)	progetto	rinvio

Test plans

Il cargoservice mantiene lo stato della stiva: il test e' **automatizzabile** (PASS/FAIL). Il caso significativo (**TP1, carico nominale**) verifica a fine ciclo: slot riservato occupato, robot a HOME,

cargoservice disponibile. Casi aggiuntivi: stiva piena (reject), IOPort occupato/Out of service (retrylater), timeout (disengage). Dettaglio in [Sprint 0 \(v1\) – Test plans](#).

Project

La traduzione del modello logico in una **FSM qak** concreta e' realizzata nel modello [cargoservice26.qak](#) (con [Hold](#), [DisplaySim](#), [LedSim](#)): il cargoservice e' un QActor client di robotsmart, struttura e build modellate su robotsmart26usage.

Testing

TP1 e' progettato per girare sul modello verificando lo stato finale della stiva. **Stato:** il modello e' scritto; generazione (plugin QAK), build ed esecuzione sono il passo successivo ([cargoservice26/README](#)).

Deployment

Vedi [Sprint 0 \(v1\) – Deployment](#): Docker (VR+GUI+mosquitto) + robotsmart26 (8020) + cargoservice26 (8030). Con mosquitto in Docker usare l'IP reale del PC.

Maintenance

Poiche' il cargoservice dipende solo dall'*informazione* scambiata (eventi logici via iosensor), sostituire i dispositivi simulati con l'hardware reale non modifica il cargoservice.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer